

PROJET ETSIA
« YANSNET – PARTIE APPLICATION »

INSTRUCTIONS D'INSTALLATION ET DE
CONFIGURATION

GROUPE PROJET 15

MAKONGNE DEFFO July Merveille
NANKAP NDIZEU Loic Aurel
NKOULOU Joseph Emmanuel
NOUNGOUA NOUNGOUA Teddy Steven
TOFA DEFFO Lionel Junior

Promotion X2027

ANNEE ACADEMIQUE 2024 – 2025

Sommaire

1. Prérequis généraux	3
1.1 Logiciels et outils requis.....	3
1.2 Comptes et accès nécessaires	3
2. Application Web – Interface Administrateur	4
2.1 Clonage du dépôt.....	4
2.2 Installation des dépendances.....	4
2.3 Configuration des variables d'environnement.....	4
2.4 Lancement en mode développement	5
2.5 Build et déploiement en production.....	5
3. Application Mobile – YANSNET App.....	5
3.1 Installation d'Expo CLI et EAS CLI	5
3.2 Clonage et installation des dépendances.....	6
3.3 Configuration de l'environnement mobile.....	6
3.4 Lancement sur émulateur ou appareil physique.....	6
3.5 Build APK / IPA pour distribution	7
4. Backend – Serveur Node.js.....	7
4.1 Installation des dépendances.....	7
4.2 Configuration de la base de données MongoDB	8
4.3 Variables d'environnement backend.....	8
4.4 Lancement du serveur	8
5. Configuration des modèles d'Intelligence Artificielle	9
5.1 Structure des fichiers modèles	9
5.2 Installation des dépendances Python	9
5.3 Vérification des modèles	9
6. Vérification globale de l'installation	10
6.1 Tests de connectivité	10

6.2 Résolution des problèmes courants.....	10
--	----

1. Prérequis généraux

1.1 Logiciels et outils requis

Avant de procéder à toute installation, assurez-vous que les logiciels suivants sont correctement installés et configurés sur votre machine de développement :

- Node.js v18 LTS ou supérieur — disponible sur nodejs.org
- npm v9+ (inclus avec Node.js) ou Yarn v1.22+
- Git v2.40+ — système de contrôle de version
- Python 3.9 ou supérieur — requis pour les modèles d'IA
- pip — gestionnaire de paquets Python (inclus avec Python 3.9+)
- MongoDB Community Server v6+ — base de données principale
- Android Studio avec Android SDK 33+ — pour l'émulateur Android
- Xcode v14+ (macOS uniquement) — pour le simulateur iOS
- Expo Go — application mobile disponible sur iOS et Android

1.2 Comptes et accès nécessaires

Les accès suivants sont indispensables pour cloner, configurer et déployer le projet :

- Accès au dépôt GitHub : https://github.com/Jozman2004/PROJET_ETZIA
- Compte Expo (expo.dev) — pour les builds EAS et le déploiement mobile
- Clé de compte de service Google Cloud (`serviceAccountKey.json`) — à récupérer auprès du responsable technique. Ce fichier ne doit jamais être versionné dans Git
- URI de connexion MongoDB (locale ou MongoDB Atlas pour la production)

2. Application Web – Interface Administrateur

2.1 Clonage du dépôt

Ouvrez un terminal et exécutez les commandes suivantes pour récupérer le code source complet du projet :

```
git clone https://github.com/Jozman2004/PROJET_ETZIA.git
cd PROJET_ETZIA/Code/site-admin/web-app/frontend
```

2.2 Installation des dépendances

Une fois dans le répertoire du frontend, installez toutes les dépendances npm déclarées dans le fichier package.json :

```
npm install
```

Cette commande installe automatiquement React, Vite, TypeScript, shadcn/ui, TanStack Table, et toutes les librairies nécessaires à l'interface administrateur.

2.3 Configuration des variables d'environnement

Le projet utilise des variables d'environnement Vite pour configurer l'URL de l'API. Procédez comme suit :

- Copiez le fichier d'exemple :

```
cp .env.example .env
```
- Ouvrez le fichier .env et renseignez les valeurs appropriées :

VITE_API_URL : http://localhost:3000 (URL du backend en développement)

En production, remplacez l'URL par l'adresse publique de votre serveur. Le fichier .env ne doit jamais être commité dans Git.

2.4 Lancement en mode développement

Pour démarrer le serveur de développement Vite avec rechargement automatique à chaud :

```
npm run dev
```

L'interface sera accessible à l'adresse : `http://localhost:5173`. Assurez-vous que le backend est également démarré pour que l'authentification et les données fonctionnent correctement.

2.5 Build et déploiement en production

Pour compiler l'application en vue d'un déploiement en production :

```
npm run build
```

Les fichiers compilés et optimisés seront générés dans le répertoire `dist/`. Pour prévisualiser le build avant déploiement : `npm run preview`. Le contenu de `dist/` peut ensuite être servi par n'importe quel serveur web statique (Nginx, Apache, Vercel, etc.).

3. Application Mobile – YANSNET App

3.1 Installation d'Expo CLI et EAS CLI

L'application mobile est développée avec React Native via le framework Expo. Installez les outils nécessaires globalement :

```
npm install -g expo-cli
```

```
npm install -g eas-cli
```

Vérifiez l'installation en contrôlant les versions installées :

```
expo --version
```

```
eas --version
```

3.2 Clonage et installation des dépendances

Naviguez vers le répertoire du frontend mobile et installez les dépendances :

```
cd PROJET_ETZIA/Code/APPLICATION-ET-BASE-DE-  
DONNEES/yansnet/frontend  
npm install
```

En cas d'erreurs de conflits de dépendances, utilisez :

```
npm install --legacy-peer-deps
```

3.3 Configuration de l'environnement mobile

Ouvrez le fichier app.json et configurez les paramètres suivants :

- `extra.apiUrl` : renseignez l'adresse IP locale de votre machine (ex : `http://192.168.1.X:3000`). N'utilisez pas « localhost » car l'émulateur ne résout pas cette adresse vers la machine hôte
- Placez le fichier `google-services.json` (Android) dans le répertoire `android/` et `GoogleService-Info.plist` (iOS) dans le répertoire `ios/` si les notifications Firebase sont activées

3.4 Lancement sur émulateur ou appareil physique

Pour démarrer le serveur Metro bundler et lancer l'application :

```
npx expo start
```

Un QR code s'affiche dans le terminal. Choisissez ensuite votre cible :

- Scannez le QR code avec Expo Go (iOS/Android) sur votre appareil physique (même réseau Wi-Fi requis)
- Touche `[a]` : lance sur l'émulateur Android (Android Studio et un AVD doivent être configurés)
- Touche `[i]` : lance sur le simulateur iOS (macOS uniquement, Xcode requis)

3.5 Build APK / IPA pour distribution

Pour générer un fichier distribuable, utilisez EAS Build. Connectez-vous d'abord :

```
eas login
```

Puis déclenchez le build selon la plateforme cible :

```
eas build --platform android  # Génère un APK ou AAB
eas build --platform ios      # Génère un IPA (Apple Developer requis)
eas build --platform all      # Génère les deux en parallèle
```

Le build est exécuté sur les serveurs Expo Cloud. Le fichier résultant est téléchargeable depuis votre tableau de bord sur expo.dev.

4. Backend – Serveur Node.js

4.1 Installation des dépendances

Le projet comporte deux backends Node.js distincts. Naviguez vers chacun et installez les dépendances :

Backend de l'interface administrateur :

```
cd PROJET_ETSIA/Code/site-admin/web-app/backend
npm install
```

Backend de l'application mobile YANSNET :

```
cd PROJET_ETSIA/Code/APPLICATION-ET-BASE-DE-
DONNEES/yansnet/backend
npm install
```

4.3 Variables d'environnement backend

Créez un fichier .env à la racine du répertoire backend et renseignez les variables suivantes :

```
PORT : 3000
MONGODB_URI : mongodb://localhost:27017/yansnet
JWT_SECRET : votre_cle_secrete_jwt_forte_et_aleatoire
JWT_EXPIRATION : 24h
GOOGLE_APPLICATION_CREDENTIALS : ./config/serviceAccountKey.json
NODE_ENV : development
```

En production, ne pas utiliser de fichier .env — utilisez les variables d'environnement système ou le gestionnaire de secrets de votre hébergeur.

4.4 Lancement du serveur

Pour démarrer en mode développement avec rechargement automatique (nodemon) :

```
npm run start:dev
```

Pour démarrer en mode production :

```
npm run build
npm run start:prod
```

Le serveur sera accessible sur `http://localhost:3000`. Vérifiez qu'il répond :

```
curl http://localhost:3000/health
```

5. Configuration des modèles d'Intelligence Artificielle

5.1 Structure des fichiers modèles

Les modèles de modération de contenu doivent être placés dans la structure suivante (répertoire Code/app/data_models/) :

```
Code/app/data_models/
├── nsfw_model/
│   └── model.safetensors    (354 Mo)
├── nsfw_model_adam/
│   └── model.safetensors    (328 Mo)
├── violence_model/
│   └── model.safetensors    (327 Mo)
└── toxic_text_model/
    └── pytorch_model.bin    (1 060 Mo)
```

Ces fichiers ne sont pas versionnés dans Git en raison de leur taille (~2 Go). Récupérez-les auprès du responsable technique ou via le lien de partage fourni séparément.

5.2 Installation des dépendances Python

Créez un environnement virtuel Python et installez les dépendances requises pour les modèles :

```
cd Code/app
python -m venv venv
venv\Scripts\activate    # Windows
source venv/bin/activate  # macOS / Linux
pip install torch torchvision torchaudio
pip install transformers safetensors Pillow
```

5.3 Vérification des modèles

Pour vérifier que les modèles sont correctement chargés et prêts à l'emploi, exécutez le script de test inclus dans le projet :

python Code/app/test_models.py

Ce script effectue un test de chargement de chaque modèle et affiche son statut. Un résultat sans erreur confirme que les modèles sont prêts à être utilisés par le backend de modération automatique.

6. Vérification globale de l'installation

6.1 Tests de connectivité

Une fois tous les composants installés et démarrés, vérifiez que chaque service répond correctement :

Backend API : `http://localhost:3000/health` → Doit retourner HTTP 200

Frontend Web : `http://localhost:5173` → Doit afficher la page de connexion

MongoDB : `mongosh` → `show dbs` → Doit lister les bases disponibles

App Mobile : `npx expo start` → Doit afficher le QR code sans erreur

Effectuez une connexion complète depuis l'interface web (login avec un compte admin) pour valider l'intégration complète entre le frontend, le backend et la base de données.

6.2 Résolution des problèmes courants

- Erreur « ECONNREFUSED 127.0.0.1:3000 » : le backend n'est pas démarré. Exécutez `npm run start:dev` dans le répertoire backend.
- Erreur « MongoServerError : Connection refused » : MongoDB n'est pas actif. Démarrez le service MongoDB avec `net start MongoDB` (Windows) ou `mongod` (macOS/Linux).
- L'application mobile ne peut pas joindre le serveur : vérifiez que l'adresse IP dans `app.json` correspond à l'IP locale de votre machine sur le réseau (pas `localhost`).

- Erreur « `import.meta.env.VITE_API_URL` is undefined » : le fichier `.env` est absent ou mal renseigné. Créez-le en copiant `.env.example` et redémarrez Vite.
- Modèle IA non trouvé au chargement : vérifiez que les fichiers `.safetensors` et `.bin` sont placés dans `Code/app/data_models/` avec la structure exacte décrite en section 5.1.
- Erreur de token JWT invalide : la valeur de `JWT_SECRET` dans le `.env` du backend doit être identique pour tous les services. Redémarrez le serveur après toute modification du `.env`.